

SAPIENZA UNIVERSITÀ DI ROMA · INTERACTIVE GRAPHICS

LUMEN DRIFT

A sci-fi exploration & platforming game, built in Three.js



Fabio Falconi

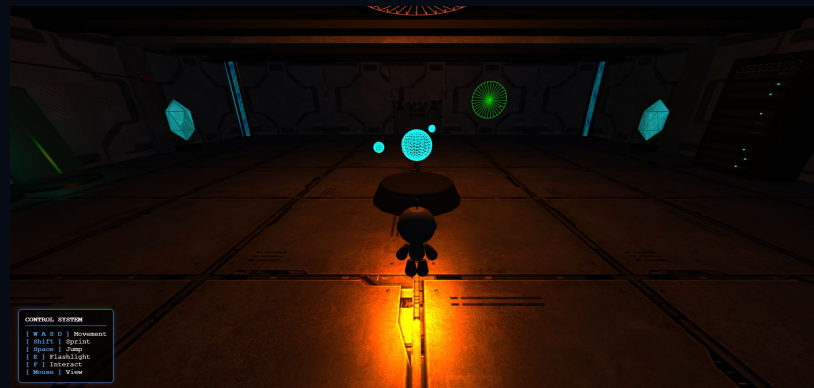
Student ID 2048079

Professor: Marco Schaerf · Academic Year 2025/2026



Introduction

- ▶ Browser-executable 3D application for the Interactive Graphics course, built entirely with Three.js and WebGL.
- ▶ The player controls an android unit in third-person view aboard a spaceship damaged by proximity to a black hole's event horizon. The player will have to recover the spaceship's reactor by solving a puzzle and face a platform section.
- ▶ In the environment background, a simulated star system orchestrates the orbital movement of six celestial bodies and their satellites in real time.
- ▶ Beyond the astronomical simulation: gravity & jump physics, a player-controlled flashlight, and light-driven environmental puzzles.



Core Libraries



Three.js (r128)

Object-oriented wrapper over WebGL; manages the full scene graph.



GLTFLoader

Loads .gltf/.glb assets — planets & structures — preserving UVs and materials.



Tween.js

Interpolates transforms; drives the main door's progressive movement.



Controls & Interaction

Movement is third-person with a self-adjusting camera. Approaching a valid object shows an on-screen “Press [F] to interact” prompt.



W A S D

Move forward, left, backward, and right relative to facing direction.



Mouse

Rotate the camera and dictate the character's facing direction.



Space

Jump over obstacles and gaps.



Shift

Hold while moving to sprint at a higher speed.



E

Toggle the android's built-in flashlight on and off.



F

Interact with terminals, buttons, and key items.



Esc

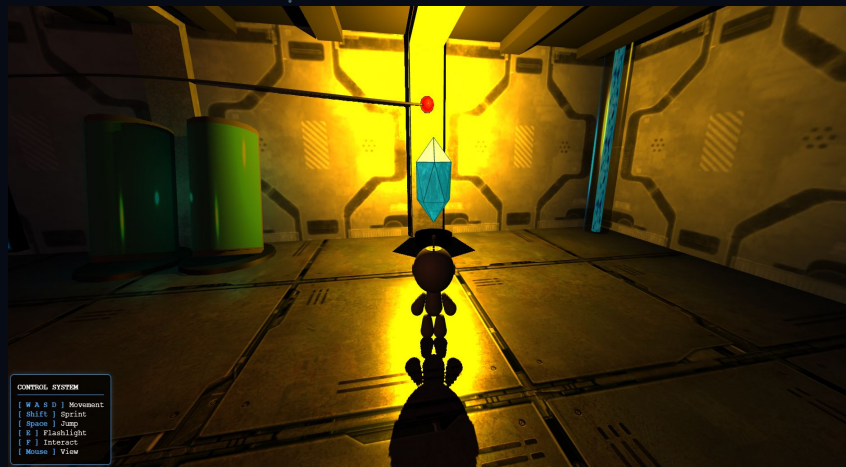
Open the pause menu, that allows to regulate the audio volume.

CONTROL SYSTEM

```
[ W A S D ] Movement  
[ Shift ] Sprint  
[ Space ] Jump  
[ E ] Flashlight  
[ F ] Interact  
[ Mouse ] View
```



The Light Puzzle



To exit the starting room, the player must overload two photo-sensitive crystal sensors at the exact same time.



Console Logs

The wall terminal hints at the solution: overload both crystals with a light beam simultaneously.



Fixed Light Source

A button activates an environmental point light aimed at the first crystal.



The Flashlight

With only one torch, the player must manually aim it (via E) at the second crystal.



The Airlock

Once both crystals register light simultaneously, the sealed main door opens.



Platforming Phase

In the external sector, the player leaps across floating wreckage. The flashlight becomes essential: platforms react dynamically to illumination.



Light-only Platforms

Invisible and lack collision by default — they materialize and become solid only when struck by a light source.



Shadow Platforms

Solid in the dark, but they vanish the instant they are illuminated by the flashlight.

The player must carefully toggle the flashlight on and off to navigate this section — eventually reaching the lost reactor in the debris field.





The Victory Scene

Once the reactor is carried back and placed on the central pedestal, the game transitions into a scripted ending cinematic.



Camera Detach

The player loses control as the camera smoothly orbits around the protagonist.



Victory Animation

The android performs a celebratory animation, swinging its arms.



Holographic UI

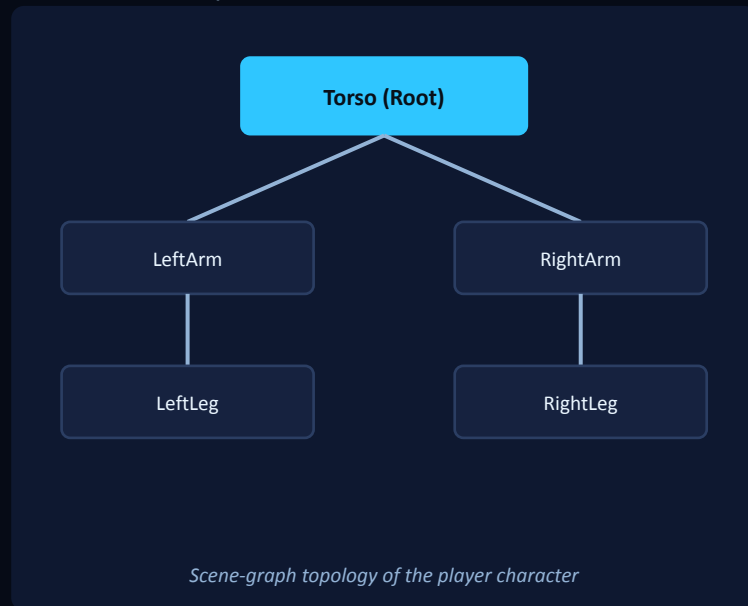
A futuristic “Mission Accomplished” panel fades into the center of the screen.





The Player: Hierarchical Model

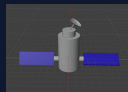
- ▶ Original monolithic mesh sourced from Sketchfab, then reorganized in Blender into five functional components: Torso (root), LeftArm, RightArm, LeftLeg, RightLeg.
- ▶ A local kinematic chain is built with Blender's Object (Keep Transform) parenting, mapped directly into the scene graph by GLTFLoader.
- ▶ Global translation/rotation on the Torso propagates natively to every limb via matrix concatenation, while each limb still animates independently around its own joint during the walk cycle.





Celestial Bodies

All major planetary assets were modeled from scratch in Blender using sculpting and material shading techniques.



Planets & Moon

UV spheres sculpted with brushes to create surface irregularities and craters. One of the standard planet has a satellite orbiting around it.



Shattered Planet

Cell-Fracture add-on splits a sphere via a Voronoi algorithm into grouped shards, lit by an internal purple point light.



Ringed Planet

A sculpted sphere paired with a flattened Torus ring using a custom gradient texture.



Volcanic Planet

Emissive magma core wrapped by a cratered crust with deleted faces and extruded volcano cones.



Glacial Planet

Pale-blue emissive sphere, Cell-Fractured for cracks, scattered with independent ice-crystal meshes.



The Comet

Asymmetrical sculpted nucleus with an attached particle system simulating its tail.

Interactive Elements



Light Sensors

Sci-fi geometry crafted in Blender. The crystalline top is a detached object, allowing the script to spin it continuously whenever its activation state updates.



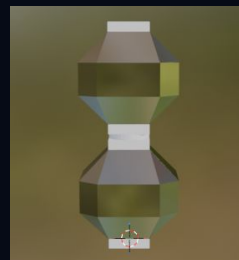
Interactive Buttons

A Sketchfab prop refactored so the movable Key mesh is isolated from the static Base — letting Tween.js interpolate the key's Z-depth for a pressing animation. Triggers a Sketchfab alarm-lamp model.



The Reactor

The mission's primary objective, modeled from scratch in Blender. Can be picked up outside the ship and carried back to the core pedestal.



Textures ship embedded inside each glTF asset. Sketchfab models keep their downloaded textures. Blender-made models use custom-authored ones.

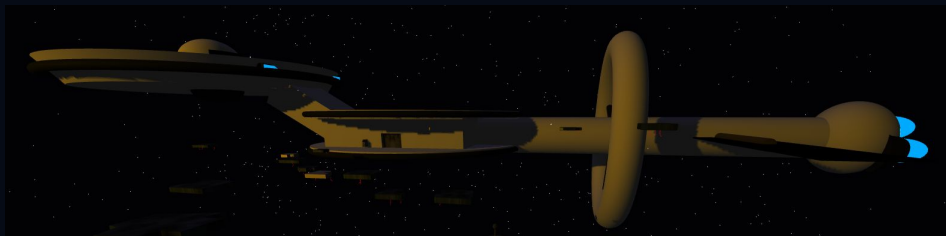


The Command Vessel & The Black Hole



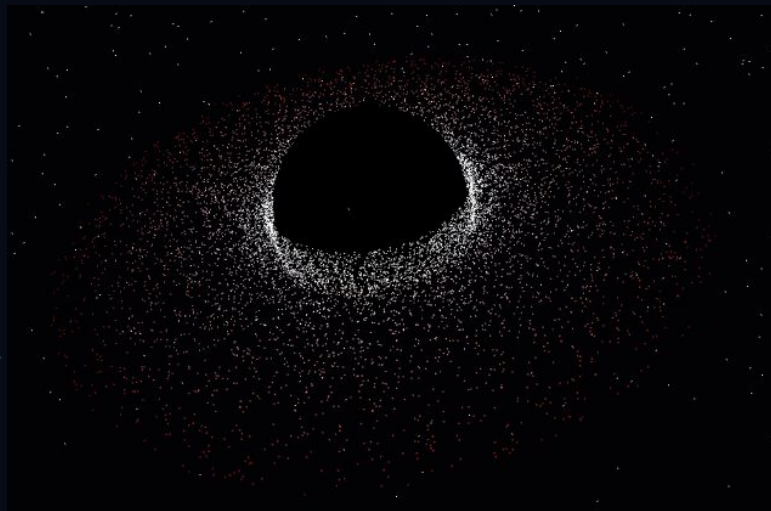
The Command Vessel

The visible hull is a hierarchical assembly of Three.js group objects composed with local transformations — a convincing massive spaceship without heavy imported meshes.



The Black Hole

The accretion disk is a THREE.BufferGeometry particle system: thousands of vertices whose position and color are computed each frame via trigonometric functions, simulating matter spiraling toward the event horizon.





Galactic Clusters & Sci-Fi Furnishings

Galactic Clusters

Each background galaxy is a point-based BufferGeometry cloud of 15,000 particles arranged on a spiral, with interpolated size and color gradients. Treated as particle clouds rather than monolithic meshes, the engine culls off-screen points and manages thousands of entities with minimal GPU pressure.



Sci-Fi Industrial Furnishings



Cryo-pods

Nested geometry simulating glass reflections.



Mainframe

Procedural hardware with periodic LED blinking.



Reactor Pedestal

Cylinder & torus altar for the reactor.



Radar

Cylindrical primitives with a scanning blip.



Hologram

Rotating wireframe solar-system display.



Wall Console

Layered box primitives for reading logs.



Character Kinematics & Orbital Motion

A single centralized `animate()` loop, synced to `requestAnimationFrame()`, drives every transform. No animation is imported.



Walk Cycle

On WASD input, a sine function of elapsed time drives limb rotation; phases invert between opposite limbs for a natural gait. The speed change if walking/running. On stop, a lerp eases every limb back to rest.



Gravity & Jump Arc

A constant gravity value continuously decreases velocityY, applied each frame to the player's Y-position until a collision is detected.



Pivot-Based Orbits

Four planets orbit the star at distinct constant angular velocities by parenting each to an invisible Pivot group rotated on Y; a moon compounds this via a secondary pivot on its parent planet.



The Binary Pair

Volcanic & Glacial planets orbit a shared barycenter. A sine wave drives both rotation speed and separation distance — an “elastic” approximation of Kepler's Second Law.





Kepler-Inspired Motion



Elliptical Comet Orbit

The orbit is an ellipse ($a = 550$, $b = 320$) on the XZ plane, offset so the star sits at a true focus rather than the center.

$$c = \sqrt{a^2 - b^2}$$

$$\omega(t) = \alpha / r(t), \quad \alpha = 20$$

$$\theta(t+\Delta t) = \theta(t) + \omega(t) \cdot 0.02$$

Orbital speed is inversely proportional to distance from the star — the comet accelerates at perihelion and slows at aphelion, a simplified Second Law. Its tail length and flutter also scale with distance, always oriented away from the sun via `lookAt()`.



Black Hole Accretion Disk

10,000 particles live in a single BufferGeometry; positions are rewritten directly in the buffer every frame instead of managing individual objects.

$$\Omega(r) = \mu / r^{1.5}, \quad \mu = 15$$

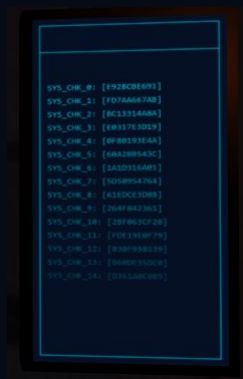
Angular speed scales inversely with radius^{1.5} — particles near the center whirl far faster than those on the outer edge, giving the disk its characteristic swirl. Each frame, positions are rotated with a 2D rotation matrix and the buffer's `needsUpdate` flag is set for the GPU.



Props animations

Many small models have animations hidden in them.

- Crystals and Platforms have oscillating floating movements
- Some materials change their opacity overtime, like background stars and The radar blip
- lights in the mainframe turn on and off
- Console log screen have moving writings
- Central Hologram rotates on itself
- Plasma pipes have moving textures to simulate moving substance inside them





Gravity, Collisions & Camera



Gravity & Fall

A constant acceleration accumulates into velocityY every frame, simulating realistic free-fall until a surface is hit.



Collision Detection

Box-intersection checks against platforms, the central hull disk, and walls lock the player's position, zero the velocity, and reset the jump state on contact.



Fail-Safe Respawn

If the player falls below a depth threshold ($Y < -30$), coordinates are reset to a safe spawn point.



Anti-Clipping Camera

A THREE.Raycaster is cast from the player toward the camera's ideal position each frame; if it hits geometry, the camera is pulled in front of the obstacle and smoothly moved into place, always looking back at the player.



The Lighting System

Three layers combine to balance a dark atmospheric aesthetic.



Global Illumination

A faint Ambient Light keeps shadows from ever being pitch black; a distant PointLight simulates the sun, lighting planets and outdoor platforming.



Local & Decorative

A shadow-free ceiling PointLight fills the main room; tiny micro-lights inside props (vents, engines) add emissive accents only.



Dynamic Flashlight

A THREE.SpotLight attached to the player, toggled on/off, is the main driver of puzzle logic — plus one interactive PointLight over a sensor, triggered by a button.

Physical Light Logic — a ray-tracing approximation

- ▶ Inverse-square attenuation from source to target, discarded early beyond a 20-unit distance cutoff for performance.
- ▶ Spotlight cone modeling via dot product against the lamp direction, remapped by the penumbra for a soft-edged falloff.
- ▶ A shadow ray (THREE.Raycaster) checks visibility, restricted only to walls, platforms, and the target itself — keeping thousands of light/object checks cheap enough to run every frame.



The Audio System

Built on native Three.js audio classes. A single AudioListener on the camera routes every sound; its master volume is set from the pause menu.



Ambient Music

A looping non-positional track plays for the whole session at low volume, unlocked on the first canvas click. Its volume is dimmed while the game is paused.



Positional Audio

A continuous buzz is a PositionalAudio source attached to the mainframe mesh: the engine scales its volume with listener distance.



Sound Effects

Short non-positional clips preloaded once and replayed on demand, restarting the sample if re-triggered in quick succession.

Event-Triggered Sound Effects

- ▶ Buttons, the sci-fi door opening and closing, the flashlight toggle, and reactor pickup and placement each fire their own cue.
- ▶ The respawn after a fall and the final victory jingle punctuate the key state transitions of the run.
- ▶ Footsteps are gated behind a timer, not played per frame. its interval shortens while sprinting, so the step rhythm matches the character's movement speed.



Thanks for your attention